Task III: Hate Speech and Offensive Language Classification

Swoyam Pokharel

Herald College Kathmandu

6CS012: Artificial Intelligence and Machine Learning

Jinu Nyachhyon

2026

Abstract

This report presents the NLP task of the final portfolio assessment. It is a three-class tweet classification task for `hate_speech`, `offensive_language`, and `neither`. The project follows the required RNN-to-LSTM workflow, then extends it with preprocessing comparisons, class weighting, a regularized LSTM follow-up, and two pretrained GloVe embeddings. All in all, this NLP task covers a total of 10 experiments.

The dataset contains `24,783` tweets and is heavily imbalanced, with `offensive_language` forming the majority class. Because of that, macro F1 is treated as the main comparison metric rather than raw accuracy alone. The best final model was `lstm_glove_twitter_50_small_regularized_weighted_lemmatized`, reaching `0.8400` accuracy, `0.7218` macro F1, and `0.8251` macro recall.

The main result is that class weighting and lemmatization improved minority-class balance, the LSTM family improved over the Simple RNN baseline, and Twitter-domain GloVe embeddings were more useful than general Wikipedia GloVe embeddings for this tweet dataset.

Keywords: text classification, RNN, LSTM, GloVe, hate speech detection, NLP

Contents

Introduction

Hate-speech and offensive-language classification is useful in content moderation systems where platforms need to triage large volumes of user-generated text before human review. The difficult part is not just detecting abusive language. The model has to separate `hate_speech`, `offensive_language`, and `neither`, which is especially difficult because the first two categories can look very similar at the token level.

The core question is: **which preprocessing and sequence-model choices improve tweet classification most effectively, especially under class imbalance?** The experiment answers that in stages. First, stemming and lemmatization are compared using Simple RNN baselines with and without class weighting. Second, the best preprocessing setup is carried into LSTM experiments. At which point, it was noticed that all of the recurrent models were showing signs of overfitting, so another model was produced specifically to tackle it: a smaller regularized LSTM with reduced capacity, `SpatialDropout1D`, recurrent dropout, and a smaller dense layer. This model performed the best, so this architecture was then reused as the base architecture for testing pretrained GloVe embeddings from general Wikipedia text and Twitter-domain text.

Finally a small inference notebook was also created to load the saved model and classify user-entered text. It is not a full Tkinter GUI, but it follows the same practical idea as the optional task in the assignment: take new text, apply the same preprocessing pipeline, and return a model prediction.

Dataset and Label Structure

The dataset contains `24,783` tweets and three labels:

- `hate_speech`,
- `offensive_language`,
- `neither`.

Figure 1

Class distribution in the assigned NLP dataset.

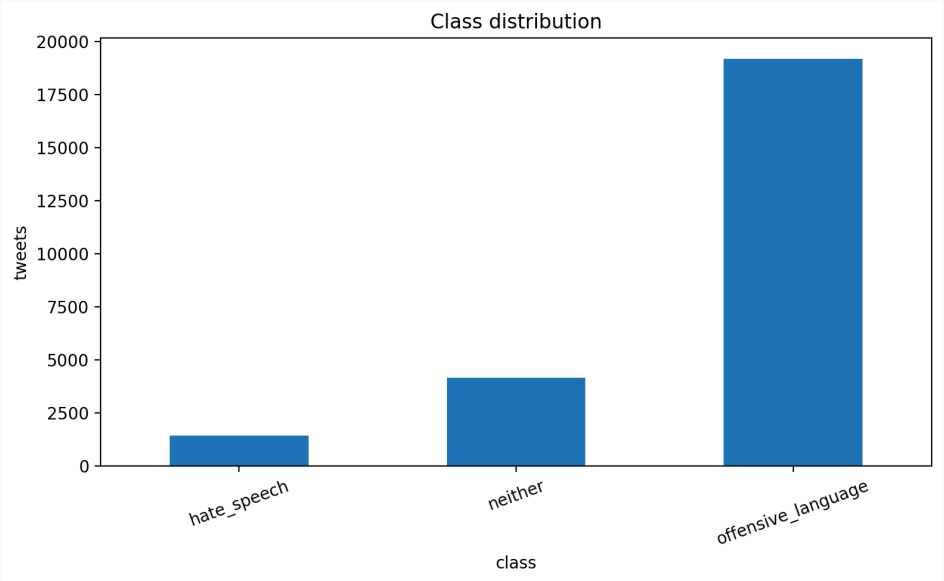


Table 1

Label distribution from `label_distribution.csv`.

Class	Tweets
offensive_language	19,190
neither	4,163
hate_speech	1,430

The imbalance is the major constraint of the task. `offensive_language` has more than thirteen times as many samples as `hate_speech`. This means raw accuracy is easy to inflate. If a model

predicts the majority class too often, it can still look decent by accuracy while failing the class that matters most for the task. Therefore macro F1, macro recall, confusion matrices, and class weighting were used as the major evaluation metrics.

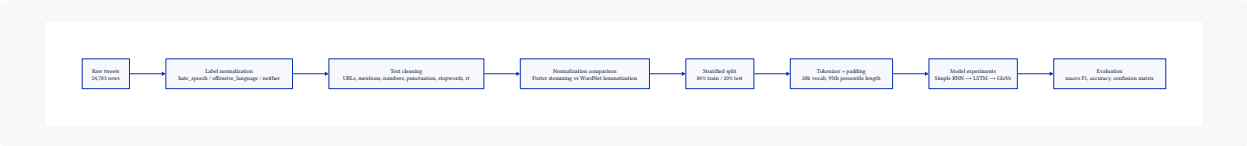
Text Preprocessing

The preprocessing pipeline cleans the tweets before tokenization. The cleaning step:

- lowercases text
- decodes HTML entities,
- expands contractions,
- removes URLs,
- removes mentions,
- removes hashtag markers,
- removes numbers,
- removes special characters,
- removes English stopwords,
- removes Twitter metadata such as `rt`.

Figure 2

Experiment Pipeline



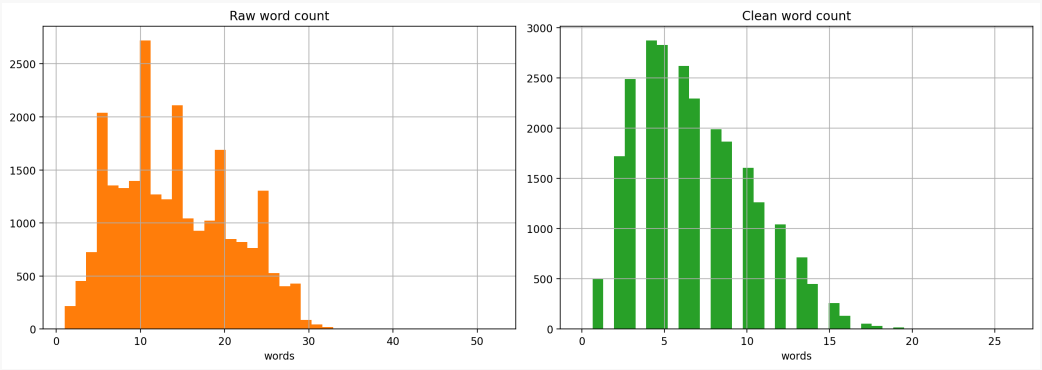
Two normalization strategies were tested:

- **Stemming:** aggressive suffix reduction using Porter stemming.
- **Lemmatization:** dictionary-based normalization using WordNet.

Stemming is cheaper and more aggressive, but it can produce distorted stems such as `studi` or `parti`. Lemmatization is cleaner because it tries to preserve actual word forms such as `study` and `party`.

Figure 3

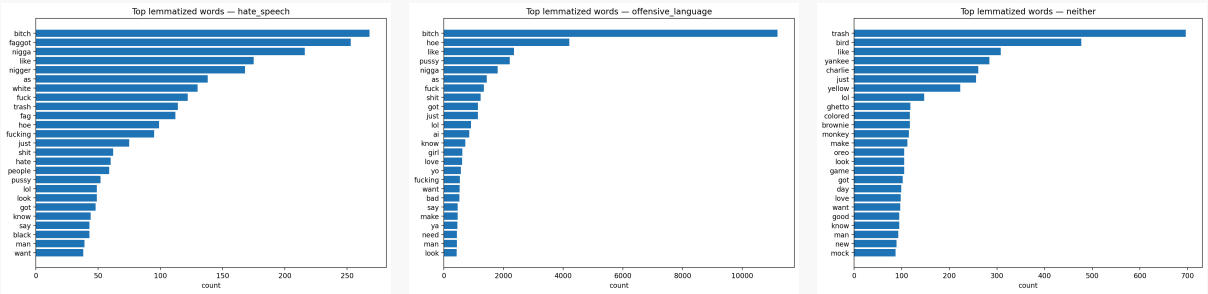
Raw versus cleaned tweet length after lemmatization.



The cleaned distribution shifts left because stopwords, mentions, punctuation, URLs, numbers, and metadata are removed. That is expected. Tweets are short already, so after cleaning many samples collapse into a small number of content-bearing words.

Figure 4

Most frequent cleaned words per class after lemmatization.



Top words: hate speech **Top words: offensive language** **Top words: neither**

Tokenization and Padding

Each preprocessing variant used its own tokenizer. The tokenizer was limited to a vocabulary size of `20,000` words and used an out-of-vocabulary token for unseen terms. Sequence length was set using the 95th percentile of cleaned training tweet length. Both the stemmed and lemmatized variants produced a maximum padded length of `13`.

Table 2

Tokenization and padding configuration.

Variant	Max length	Vocabulary seen
Stemmed	13	13,675
Lemmatized	13	15,945

The dataset was split using a stratified `80/20` train-test split. Stratification matters here because the class imbalance is large. Without it, the test set could accidentally contain a distorted class distribution, making model comparisons less defensible.

Experimental Setup

All experiments were implemented in TensorFlow/Keras with a fixed seed of 42. The code was split into Python modules rather than being placed entirely inside the notebook.

```
src/nlp/
├── data.py          # load dataset, encode labels, and split data
├── text.py          # clean text, normalize, tokenize, and pad
├── eda.py           # label plots, text-length plots, and word freq
├── models.py        # RNN, LSTM, BiLSTM, and regularized model builders
├── embeddings.py    # GloVe loading and embedding-matrix creation
├── experiments.py   # train, evaluate, save metrics and models
└── inference.py     # saved-model inference helpers

outputs/
├── figures/nlp/         # EDA plots, curves, comparisons
├── histories/nlp/       # per-experiment training history files
├── model_summaries/nlp/ # saved model architecture summaries
├── models/nlp/          # saved trained Keras models
├── predictions/nlp/     # per-sample prediction files
└── tables/nlp/          # metrics, reports, processed splits
```

Every model's results were stored under `outputs/`. This included training histories, classification reports, confusion matrices, prediction files, model summaries, saved models, tokenizer files, and cleaned train-test splits. This was done to keep the report based on saved evidence rather than notebook memory or manually copied screenshots.

Simple RNN Experiment Grid

The first modelling stage used Simple RNNs to answer two basic questions before moving to LSTM models:

1. Should the pipeline use stemming or lemmatization?
2. Should class weights be used to handle the imbalance?

This produced four Simple RNN experiments:

- `simple_rnn_stemmed`
- `simple_rnn_weighted_stemmed`
- `simple_rnn_lemmatized`
- `simple_rnn_weighted_lemmatized`

Figure 5

Experiment progression from Simple RNN to LSTM and GloVe-based models.

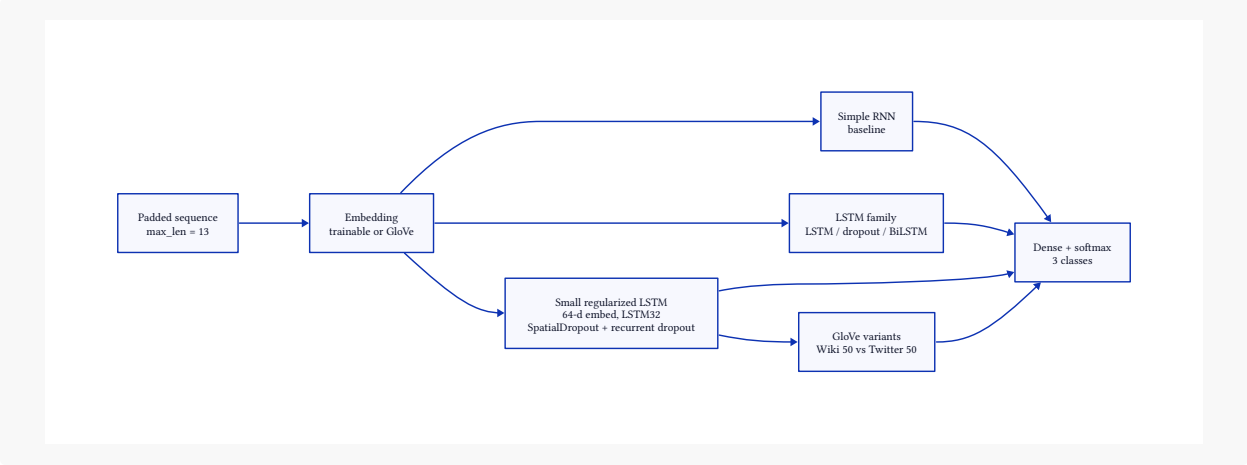
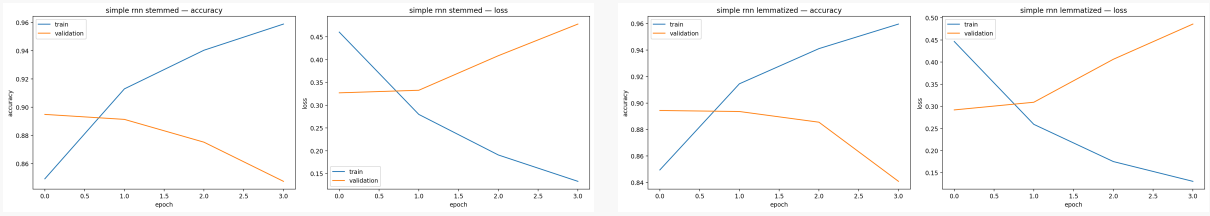


Figure 6

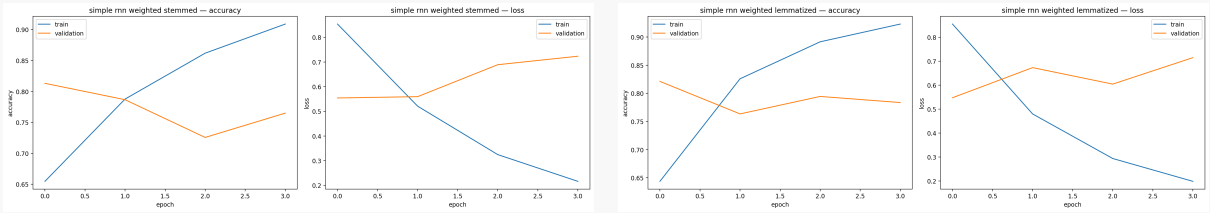
Training curves for unweighted Simple RNN models.



Simple RNN + stemming **Simple RNN + lemmatization**

Figure 7

Training curves for class-weighted Simple RNN models.



Weighted Simple RNN + stemming **Weighted Simple RNN + lemmatization**

Figures 6 and 7 show the first major problem. The training accuracy keeps improving and the training loss keeps falling, but the validation curves do not follow the same pattern. In the unweighted models, validation accuracy drops while validation loss rises, which is a direct overfitting signal. The class-weighted models are noisier as they are forced to pay more attention to the minority classes, but they still show the same general issue: the Simple RNN can fit the training data faster than it can generalize.

Figure 8

Macro F1 comparison for the Simple RNN experiment grid.

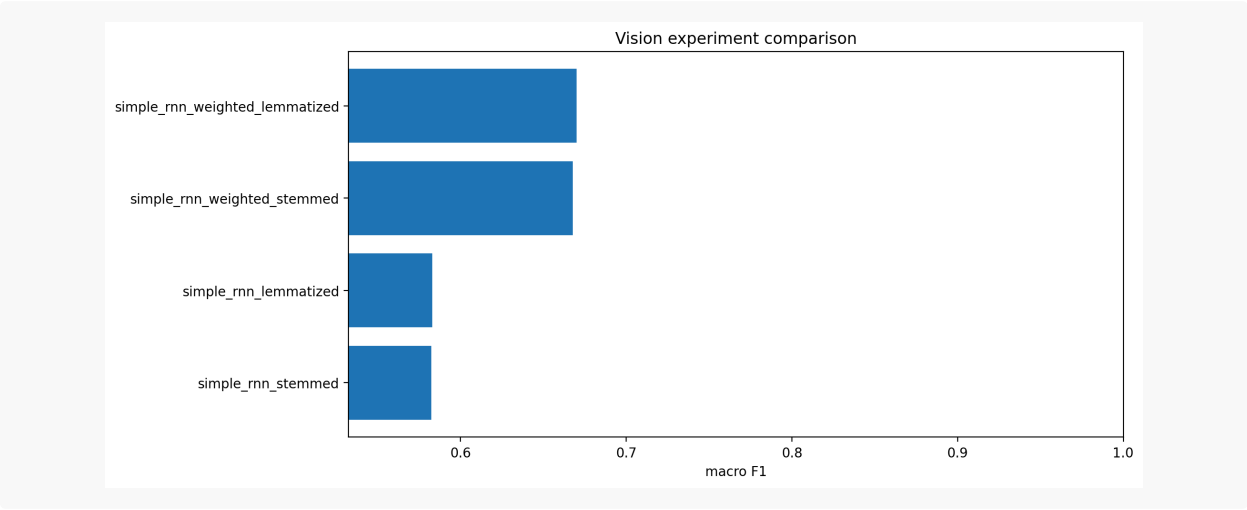


Table 3

Simple RNN comparison.

Model	Acc.	Macro F1	Macro Recall	Errors	Epochs	Params
Weighted + Lemma	0.8104	0.6702	0.7540	940	4	2.57M
Weighted + Stem	0.8065	0.6677	0.7512	959	4	2.57M
Lemma	0.8866	0.5829	0.5998	562	4	2.57M
Stem	0.8872	0.5825	0.5963	559	4	2.57M

The unweighted Simple RNN models had higher accuracy, but their macro F1 was much weaker. This is the imbalance problem showing up directly. The model can reduce total mistakes by simply focusing on the majority class, but that does not mean it is handling the other classes well. The higher accuracy is a bit misleading here. The unweighted models look stronger by raw accuracy, but they are mostly benefiting from the majority `offensive_language` class. The weighted models sacrifice some accuracy, but they produce better macro F1 and macro recall, which is more important for this imbalanced classification task.

Thus, the best Simple RNN configuration was `simple_rnn_weighted_lemmatized`, with `0.6702` macro F1 and `0.7540` macro recall. This result decides the next stage, lemmatization and class weighting are carried forward into the LSTM experiments.

LSTM Experiments

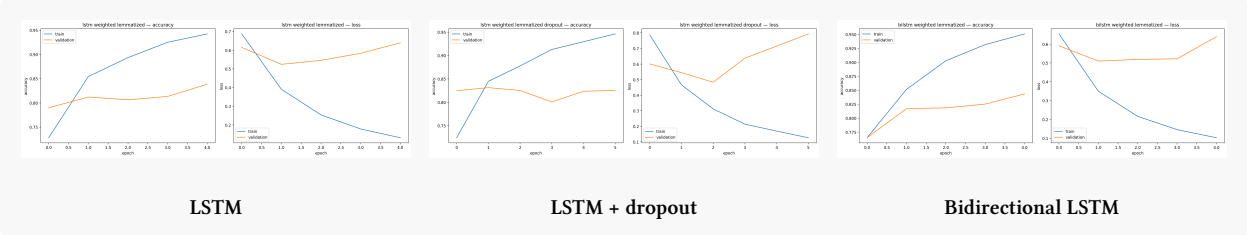
The LSTM stage keeps the winning Simple RNN setup fixed: lemmatized text and class weights. The goal is to test whether a stronger recurrent architecture improves sequence modelling compared with Simple RNN.

The initial LSTM experiments were:

- `lstm_weighted_lemmatized`
- `lstm_weighted_lemmatized_dropout`
- `bilstm_weighted_lemmatized`

Figure 9

Training curves for LSTM-family experiments.



LSTM **LSTM + dropout** **Bidirectional LSTM**

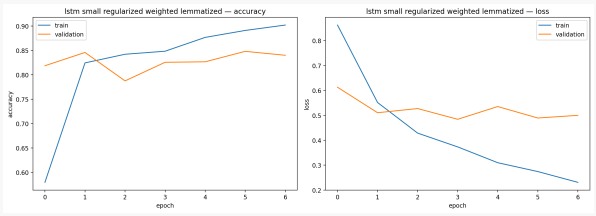
The LSTM models improved over the Simple RNN baseline, but the training curves still showed overfitting. Training loss kept dropping while validation loss rose after the early epochs. That means the model was learning the training set faster than it was improving generalization.

So, to counter this overfitting problem, another model was added to address this directly: `lstm_small_regularized_weighted_lemmatized`. It reduced the embedding dimension, used fewer LSTM units, added `SpatialDropout1D`, used recurrent dropout inside the LSTM, and used

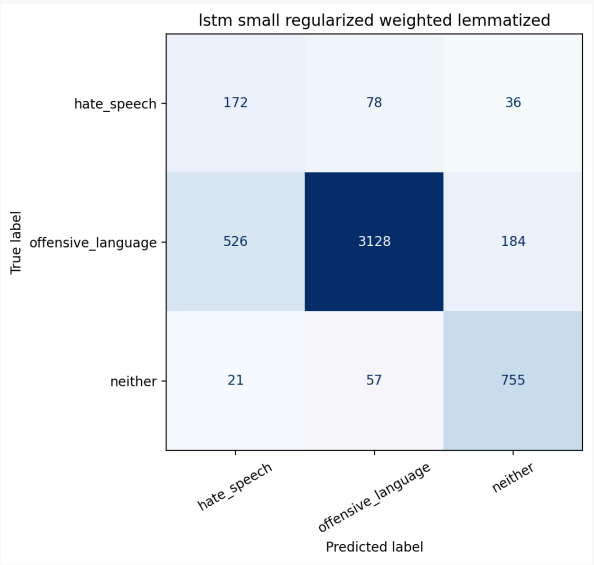
a smaller dense layer. This was not a random extra model but aimed to be a direct response to the overfitting pattern.

Figure 10

Regularized LSTM follow-up after observing overfitting.



Small regularized LSTM



Small regularized LSTM confusion matrix

Figure 10 shows that the regularized LSTM did not completely remove overfitting, but it reduced the gap enough to make the model more useful. The validation curve is still not perfectly smooth, which is expected with a small and imbalanced tweet dataset, but the model no longer behaves like a larger LSTM that simply keeps fitting the training set. The confusion matrix also shows the remaining bottleneck: many `hate_speech` samples are still being classified as `offensive_language`, which makes sense because those two labels are semantically close.

Figure 11

Macro F1 comparison for Simple RNN and LSTM-family models.

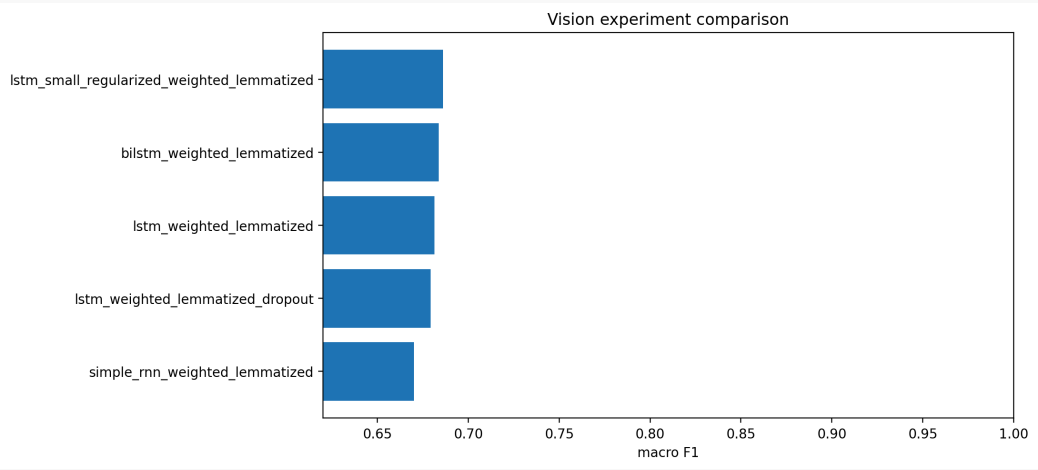


Figure 11 shows the actual result of this design choice. The smaller regularized LSTM became the strongest trainable-embedding model by macro F1, beating the larger LSTM, the dropout LSTM, the BiLSTM, and the best Simple RNN baseline. So, reducing capacity and adding regularization did not weaken the model here. It improved the balance between learning the sequence patterns and not overfitting the training set.

Table 4

LSTM-family comparison.

Model	Acc.	Macro F1	Macro Recall	Errors	Epochs	Params
Small Reg. LSTM	0.8180	0.6862	0.7743	902	7	1.29M
BiLSTM	0.8094	0.6837	0.7837	945	5	2.66M
LSTM	0.7997	0.6815	0.7852	993	5	2.61M
LSTM + Dropout	0.8114	0.6792	0.7640	935	6	2.61M
Simple RNN	0.8104	0.6702	0.7540	940	4	2.57M

The small regularized LSTM became the best trainable-embedding model, reaching `0.6862` macro F1, `0.8180` accuracy, and `0.8422` weighted F1. It also used only `1.29M` trainable parameters, roughly half the size of the larger LSTM and BiLSTM models.

Pretrained GloVe Embedding Experiments

The assignment requires pretrained Word2Vec-style embeddings. The appendix uses GloVe through Gensim, so the pretrained stage uses GloVe embeddings. The best trainable-embedding architecture from the LSTM stage was reused: the small regularized weighted LSTM.

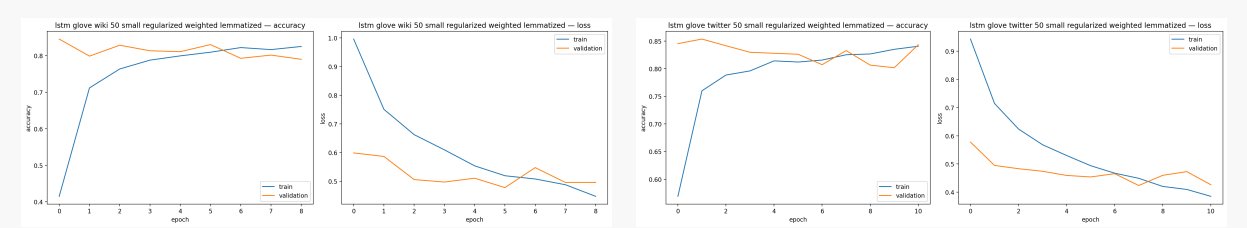
Two embedding sources were compared:

- `glove-wiki-gigaword-50`: general Wikipedia/news-style GloVe.
- `glove-twitter-50`: Twitter-domain GloVe.

This comparison was chosen because the dataset is made of tweets. A Twitter-trained embedding is more likely to represent social-media spelling, slang, abbreviations, and informal word usage than a general Wikipedia/news embedding. The experiment therefore tests the hypothesis: pre-trained embeddings should work better when their training domain is closer to the target dataset.

Figure 12

Training curves for pretrained GloVe embedding experiments.

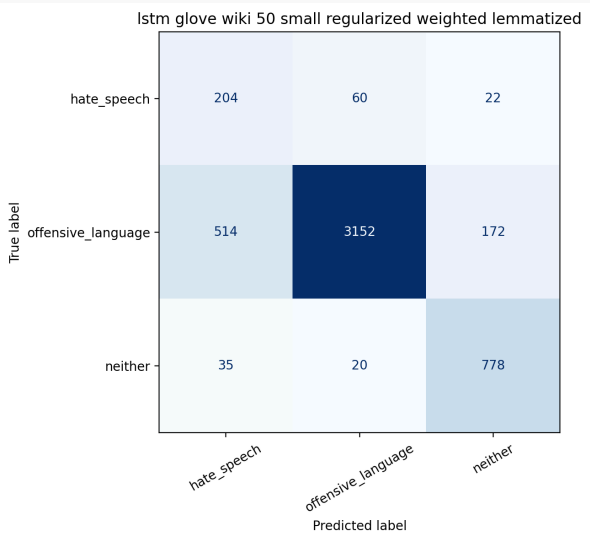


GloVe Wiki 50 **GloVe Twitter 50**

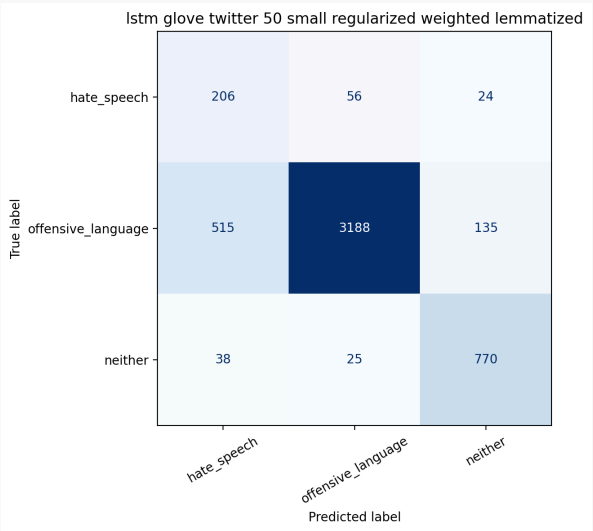
This graph shows an important shift. The pretrained GloVe models reduce the overfitting pattern seen in the trainable-embedding LSTM because the embedding layer no longer has to learn word meaning from this relatively small and imbalanced dataset alone. The model starts with useful semantic structure from a much larger external corpus, so training can focus more on the classification boundary instead of spending capacity learning basic word relationships from scratch. That is why the validation curves look more stable compared with the earlier LSTM variants.

Figure 13

Confusion matrices for pretrained GloVe embedding experiments.



GloVe Wiki 50 confusion matrix



GloVe Twitter 50 confusion matrix

However, Figure 13 shows that the main problem is not fully solved. Both GloVe models still confuse many `hate_speech` samples with `offensive_language`, because those labels are close in actual language use. However, the Twitter GloVe model reduces the total number of mistakes and gives the strongest macro F1, which supports the hypothesis: tweet-trained embeddings transfer better to tweet classification than general Wikipedia/news embeddings.

Figure 14

Macro F1 comparison for trainable embedding, Wiki GloVe, and Twitter GloVe models.

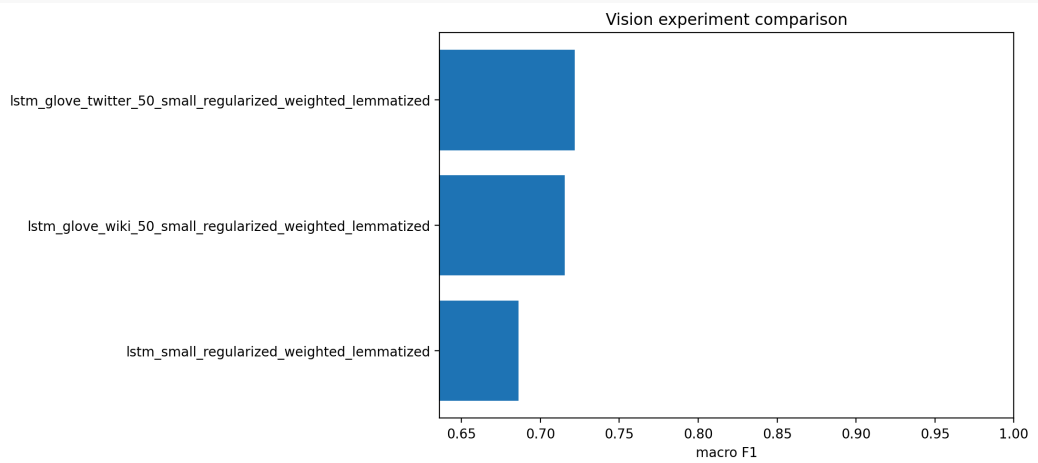


Table 5

Pretrained embedding comparison.

Model	Acc.	Macro F1	Macro Recall	Errors	Epochs	Params
GloVe Twitter 50	0.8400	0.7218	0.8251	793	11	1.01M
GloVe Wiki 50	0.8340	0.7155	0.8228	823	9	1.01M
Trainable Embed.	0.8180	0.6862	0.7743	902	7	1.29M

Both pretrained embedding models improved over the trainable-embedding regularized LSTM. The Twitter-domain embedding performed best overall, reaching `0.8400` accuracy, `0.7218` macro F1, and `0.8251` macro recall. The Wiki GloVe model was also strong, but slightly weaker. This supports the domain-match argument: for tweet classification, Twitter-trained word vectors are more useful than general Wikipedia/news vectors.

Final Comparison

Figure 15

Final macro F1 ranking across all NLP experiments.

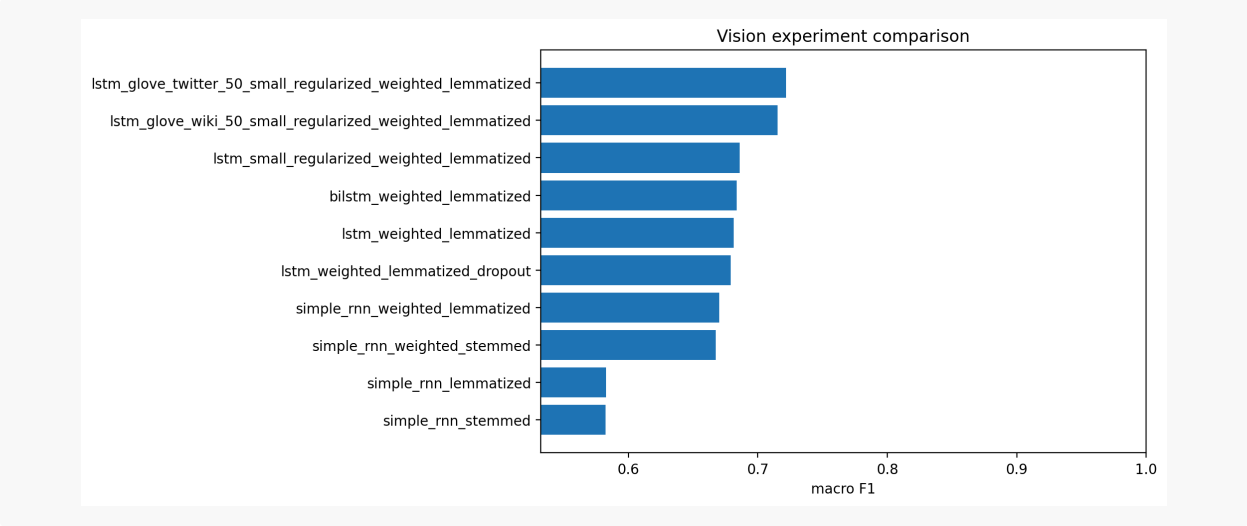


Table 6

Top five NLP models ranked by macro F1.

Rank	Model	Accuracy	Macro F1	Errors
1	GloVe Twitter Small Reg. LSTM	0.8400	0.7218	793
2	GloVe Wiki Small Reg. LSTM	0.8340	0.7155	823
3	Small Reg. LSTM	0.8180	0.6862	902
4	BiLSTM	0.8094	0.6837	945
5	LSTM	0.7997	0.6815	993

The final ranking is clear. The strongest model was the lemmatized, class-weighted, regularized LSTM using `glove-twitter-50`: `lstm_glove_twitter_50_small_regularized_weighted_lemmatized`. It improved over the best Simple RNN from `0.6702` to `0.7218` macro F1, and reduced misclassified samples from `940` to `793`.

So, all in all the total improvements came from a chain of decisions:

- Class weighting improved macro F1 even when it reduced raw accuracy.
- Lemmatization slightly outperformed stemming under the class-weighted Simple RNN setup.
- LSTM models improved over Simple RNN, but also exposed a clear overfitting problem.

- The smaller regularized LSTM handled that overfitting better than the larger recurrent variants.
- Pretrained GloVe embeddings improved the best regularized LSTM by giving the model stronger word representations from the start.
- Twitter GloVe outperformed Wiki GloVe because the embedding source matched the tweet dataset more closely.

Error Analysis and Inference Interface

The final model still makes a pleanty mistakes. The biggest challenge that it has to separate `hate_speech` from `offensive_language`, which is a hard boundary because both classes can contain aggressive or abusive language. The model improves minority-class recall compared with the unweighted Simple RNN, but the confusion matrices still show that perfect separation is not realistic for this dataset.

Additionally, the project also includes a lightweight inference notebook in the same repo: `2431342_Swoyam_Pokharel_NLP_Inference.ipynb`. It loads the saved best model, loads the lemmatized tokenizer, applies the same cleaning and padding pipeline, and returns class probabilities for user-entered text. This satisfies the real-time prediction requirement without adding a separate GUI framework. The mechanism is the same as deployment: clean text, tokenize, pad, run forward prediction, return probabilities.

Model Complexity and Practical Trade-offs

The strongest final model used pretrained Twitter GloVe embeddings with the small regularized LSTM architecture. It had `1,011,203` trainable parameters, compared with `2,574,531` for the Simple RNN models, `2,611,587` for the larger LSTM variants, `2,663,043` for the BiLSTM, and `1,292,995` for the trainable-embedding small regularized LSTM. The best model was therefore

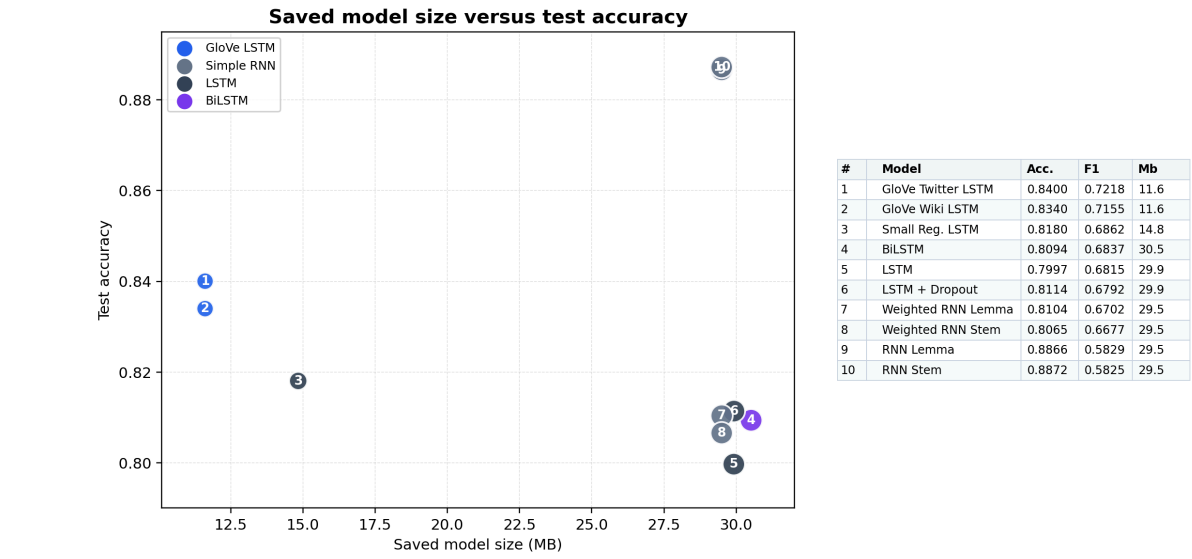
not the largest model. It was the model with the better representation source and the better regularization balance.

The trade-off is straightforward:

- If the goal is maximum raw accuracy, the unweighted Simple RNN looks strongest, but this is misleading because macro F1 is weak.
- If the goal is balanced class performance, the Twitter GloVe regularized LSTM is the better option.
- If the goal is fewer trainable parameters, the GloVe models are also efficient because they use stronger pretrained word representations instead of learning everything from scratch.

Figure 16

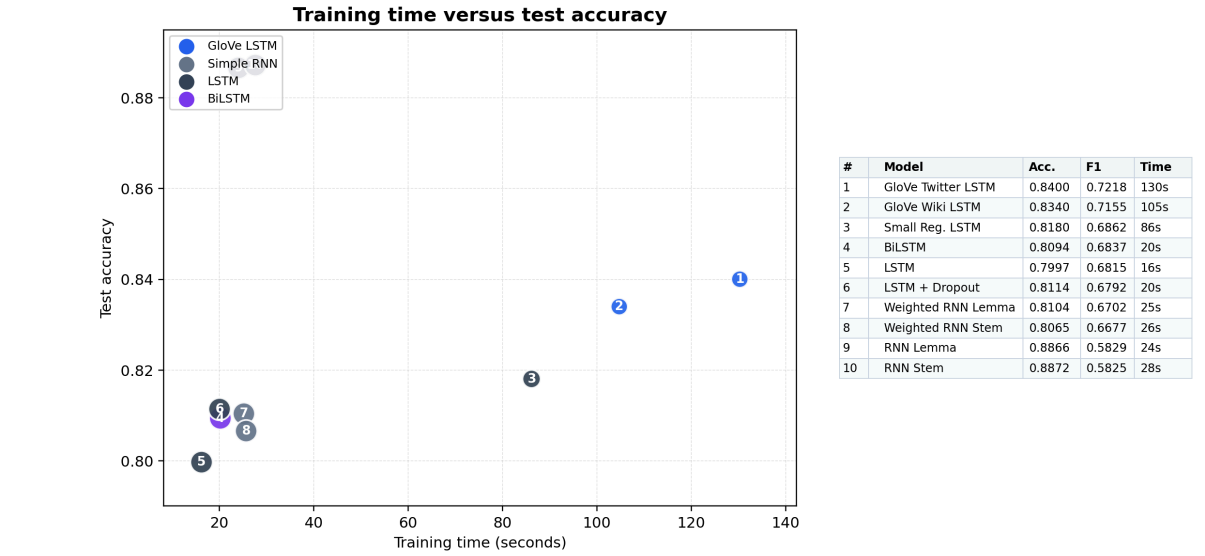
Saved model size versus test accuracy for NLP experiments.



#	Model	Acc.	F1	Mb
1	GloVe Twitter LSTM	0.8400	0.7218	11.6
2	GloVe Wiki LSTM	0.8340	0.7155	11.6
3	Small Reg. LSTM	0.8180	0.6862	14.8
4	BiLSTM	0.8094	0.6837	30.5
5	LSTM	0.7997	0.6815	29.9
6	LSTM + Dropout	0.8114	0.6792	29.9
7	Weighted RNN Lemma	0.8104	0.6702	29.5
8	Weighted RNN Stem	0.8065	0.6677	29.5
9	RNN Lemma	0.8866	0.5829	29.5
10	RNN Stem	0.8872	0.5825	29.5

Figure 17

Training time versus test accuracy for NLP experiments.



#	Model	Acc.	F1	Time
1	GloVe Twitter LSTM	0.8400	0.7218	130s
2	GloVe Wiki LSTM	0.8340	0.7155	105s
3	Small Reg. LSTM	0.8180	0.6862	86s
4	BiLSTM	0.8094	0.6837	20s
5	LSTM	0.7997	0.6815	16s
6	LSTM + Dropout	0.8114	0.6792	20s
7	Weighted RNN Lemma	0.8104	0.6702	25s
8	Weighted RNN Stem	0.8065	0.6677	26s
9	RNN Lemma	0.8866	0.5829	24s
10	RNN Stem	0.8872	0.5825	28s

Figures 16 and 17 show why accuracy alone is not enough to judge the engineering trade-off. The unweighted Simple RNN models are high on accuracy, but they are not the best practical models because they underperform on macro F1. The GloVe-based LSTM models give up some raw accuracy, but they improve the metric that matters more for this imbalanced task.

Training time also does not map cleanly to better performance. The best model takes longer than the basic LSTM models, but the extra cost buys better class balance rather than just a higher accuracy number. The final choice therefore depends on what is being optimized: raw accuracy, balanced minority-class performance, or a smaller model with stronger pretrained representations. For this task, the strongest compromise is the Twitter GloVe regularized LSTM.

Limitations

The experiments have several limitations. First, the dataset is heavily imbalanced, so the final model still depends on class weighting and macro-level evaluation. The `hate_speech` class has far fewer examples than the majority class, making it harder to learn cleanly. This also means

that high accuracy should be interpreted carefully, because accuracy can hide weak minority-class behaviour.

Another limitation is that the model only sees the cleaned tweet text, not the broader social context around the post. This matters because `offensive_language` and `hate_speech` can share similar vocabulary, while the actual distinction may depend on target, intent, or context. The model can learn useful lexical and sequence patterns, but it is still limited by how much of that distinction is visible in the text alone.

Finally, the pretrained embedding comparison only used 50-dimensional GloVe variants. Larger embeddings may perform differently, but they would also increase download size, training time, and computational cost.

Conclusion

The NLP experiments showed that class weighting and lemmatization were the strongest baseline setup when macro F1 was prioritized. The LSTM experiments improved sequence modelling but also exposed overfitting, which led to the smaller regularized LSTM follow-up. The pretrained embedding experiments then produced the strongest result, with Twitter GloVe outperforming Wiki GloVe.

The final model was `lstm_glove_twitter_50_small_regularized_weighted_lemmatized`, reaching `0.8400` accuracy, `0.7218` macro F1, and `0.8251` macro recall.